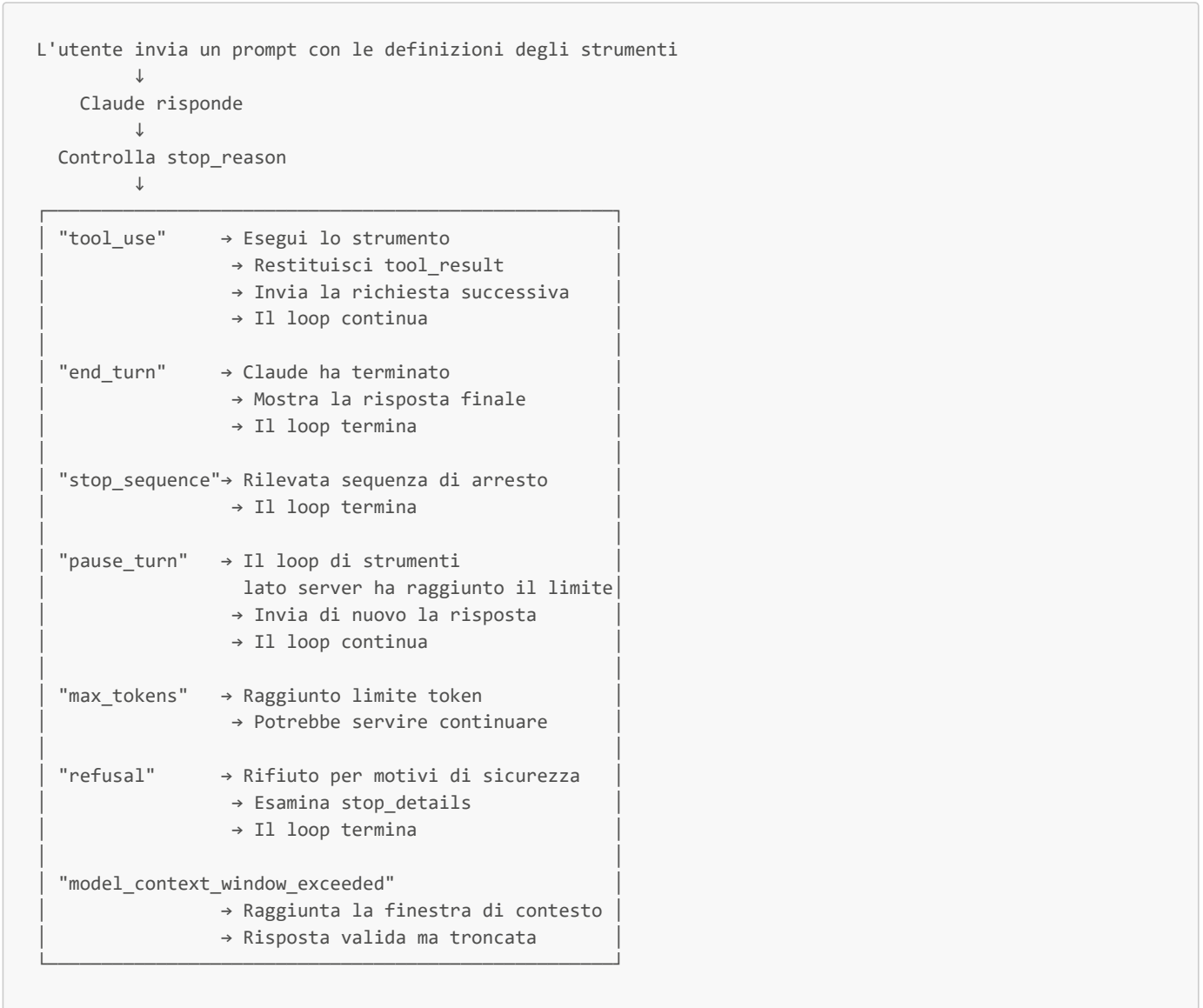
Dominio 1: architettura agentica e orchestrazione (27%)

È il dominio con il peso maggiore nell'esame. Comprende 7 obiettivi incentrati su come si costruiscono i sistemi agentici, come funzionano i cicli, come si coordinano gli agenti e come si applicano i controlli.

1.1 Ciclo di vita del loop agentico

In sostanza, un sistema agentico è un loop. Claude riceve un prompt con gli strumenti disponibili, decide se chiamare uno strumento o rispondere e il loop continua finché Claude non segnala di aver terminato.

Come funziona il loop



Valori di `stop_reason`

Valore	Significato	Azione
`"tool_use"`	Claude vuole chiamare uno strumento	Eseguilo, restituisci il risultato e continua il loop
`"end_turn"`	Claude ha terminato il ragionamento	Mostra la risposta testuale finale
`"stop_sequence"`	L'output corrisponde a una stringa configurata in `stop_sequences`	Il loop termina; controlla quale sequenza è stata rilevata

Valore	Significato	Azione
`"pause_turn"`	Il loop di strumenti lato server (ad esempio `web_search`) ha raggiunto il proprio limite interno di iterazioni	Invia di nuovo la risposta per continuare
`"max_tokens"`	La risposta ha raggiunto il limite di `max_tokens`	Potrebbe essere necessario richiedere una continuazione
`"refusal"`	Claude ha rifiutato di rispondere per motivi di sicurezza	Il loop termina; esamina `stop_details` (vedi sotto), riformula o reindirizza la richiesta
`"model_context_window_exceeded"`	La generazione ha raggiunto la finestra di contesto del modello prima di `max_tokens`	La risposta è valida ma troncata; riduci l'input o continua. Predefinito da Sonnet 4.5 in poi

> `stop_details` **nei rifiuti (Opus 4.7+):** una risposta `refusal` include anche un oggetto `stop_details` (non serve un'intestazione beta). `stop_details.type` è sempre `"refusal"`; `stop_details.category` indica la categoria della policy (`"cyber"`, `"bio"`, `"frontier_llm"`, `"reasoning_extraction"`, `"general_harms"` oppure `null`; l'insieme può ampliarsi nel tempo); `stop_details.explanation` è una stringa leggibile da una persona (non analizzarla programmaticamente). `stop_details` è `null` per ogni altro motivo di arresto. Usa la categoria per instradare o registrare in modo diverso i rifiuti specifici. Dal 2026-09-24, un rifiuto ricevuto **prima che venga addebitato qualsiasi output** è gratuito quando la categoria è `bio`, `frontier_llm` o `reasoning_extraction` (le categorie con pochi falsi positivi); i rifiuti `cyber`, `general_harms` e `null` ricevuti prima dell'output restano comunque gratuiti. I rifiuti a generazione avviata sono sempre stati addebitati e ogni rifiuto conta ai fini dei limiti di frequenza.

Fallback lato server in caso di rifiuto

Un `refusal` arriva con **HTTP 200**, non come eccezione: il codice che legge `content` senza controllare prima `stop_reason` elaborerà silenziosamente una risposta vuota o parziale. Per gli agenti in produzione basati su Claude Opus 5.5, Opus 5 e Fable 5 / 5.1 (tutti e quattro includono classificatori di sicurezza; le protezioni di Mythos 5.1 dipendono dal programma di accesso, quindi gestisci anche lì `refusal`), non limitarti a registrare il rifiuto: attiva il fallback lato server, così la richiesta viene reindirizzata automaticamente in base alla categoria del rifiuto.

```
response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=16000,
    betas=["server-side-fallback-2026-07-01"],
    fallbacks="default",          # il server instrada in base alla categoria del rifiuto
    messages=[...],
)
```

Con `fallbacks="default"` non devi mantenere un elenco di modelli. Funziona ancora anche la precedente forma con array (`betas=["server-side-fallback-2026-06-01"]` + `fallbacks=[{"model": "claude-opus-4-8"}]`). Il fallback lato server è disponibile **solo con Claude API**: su Bedrock, Vertex e Foundry usa invece il middleware lato client `BetaRefusalFallbackMiddleware` degli SDK.

Una conseguenza per Fable 5.1 e Opus 5.5 è che il fallback spesso passa a un modello che non è in grado di leggere i relativi blocchi di ragionamento. Fable 5.1 / Mythos 5.1 leggono tutto (i blocchi dell'altro, quelli di Opus 5.5 e quelli dei modelli precedenti); Opus 5.5 legge i blocchi di Opus 5 e dei precedenti Opus / Sonnet / Haiku, ma non quelli di Fable o Mythos; nessun modello precedente legge i blocchi di Fable 5.1 o Opus 5.5. L'API elimina i blocchi non leggibili (senza addebito), la richiesta va a buon fine e il modello di fallback pianifica di nuovo senza quel ragionamento: dopo il cambio, aspettati un primo turno più lento e costoso. Invia l'intestazione beta `thinking-binding-controls-2026-08-01` per ricevere un elenco `input_transformations` dei contenuti eliminati.

Regola del loop: dirama sempre il flusso in base a `stop_reason` *prima* di leggere `content`. `refusal` è un motivo di arresto definitivo per quella richiesta, non un errore dello strumento da ritentare. Inoltre, con Fable 5.1 e Opus 5.5, l'array `messages` deve essere **solo in aggiunta**: modificare o rimuovere turni precedenti invalida tutti i successivi blocchi di ragionamento (Dominio 5 §5.7).

Struttura della risposta API

Quando Claude vuole usare uno strumento, la risposta contiene sia testo sia un blocco `tool_use`:

```
{
  "id": "msg_01Aq9w938a90dw8q",
  "model": "claude-opus-5-5",
  "stop_reason": "tool_use",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Controllo il meteo attuale a San Francisco."
    },
    {
      "type": "tool_use",
      "id": "toolu_01A09q90qw90lq917835lq9",
      "name": "get_weather",
      "input": { "location": "San Francisco, CA", "unit": "celsius" }
    }
  ]
}
```

L'array `content` può contenere più blocchi: nella stessa risposta Claude può spiegare cosa sta facendo (testo) e chiamare uno strumento (`tool_use`). Può anche chiamare più strumenti contemporaneamente includendo più blocchi `tool_use`.

Restituzione dei risultati degli strumenti

I risultati degli strumenti devono fare riferimento all'esatto `tool_use_id` della richiesta di Claude:

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01A09q90qw90lq917835lq9",
      "content": "15 gradi Celsius, parzialmente nuvoloso"
    }
  ]
}
```

Per più chiamate simultanee, restituisci più blocchi `tool_result` nello stesso messaggio, ciascuno associato al rispettivo `tool_use_id`.

Risultati di errore

Quando l'esecuzione di uno strumento non riesce:

```
{
  "type": "tool_result",
  "tool_use_id": "toolu_01A09q90qw90lq917835lq9",
  "is_error": true,
  "content": "Timeout dell'API dopo 30 secondi"
}
```

Impostare `is_error: true` aiuta Claude a capire che lo strumento non è riuscito e a valutare come recuperare.

Decisioni guidate dal modello

L'esame verifica che tu abbia compreso questo concetto: **è Claude a decidere** quale strumento chiamare e quando fermarsi. Non si preconfigurano alberi decisionali o sequenze fisse di strumenti. Il modello valuta quali informazioni gli servono, seleziona lo strumento appropriato, elabora il risultato e decide il passo successivo.

È un approccio fondamentalmente diverso dall'automazione tradizionale dei flussi di lavoro, in cui i passaggi sono prestabiliti.

Antipattern (risposte errate)

1. **Analizzare il linguaggio naturale** per terminare il loop: «Se Claude dice "ho finito", esci dal loop». Errato. Controlla sempre `stop_reason`.
2. **Usare limiti arbitrari di iterazioni** come criterio principale di arresto: «Esegui al massimo 5 cicli e poi fermati». Errato come meccanismo principale. Puoi impostare limiti di sicurezza, ma è `stop_reason` a guidare il loop.
3. **Controllare la presenza di testo** per stabilire che l'elaborazione è finita: «Se la risposta contiene testo, ha terminato». Errato. Le risposte possono contenere sia testo sia blocchi `tool_use`.

> **Oltre la guida d'esame: budget delle attività (beta):** la soluzione moderna alla domanda «come limito un loop agentico senza imporre un numero massimo di iterazioni» è un **budget dell'attività**. `output_config.task_budget` assegna a Claude un limite di token che può *vedere*, così può dosare il lavoro e concluderlo in modo ordinato invece di essere interrotto a metà. È diverso da `max_tokens`, che impone un limite per risposta di cui il modello non è consapevole.
>
> ```
> with client.beta.messages.stream(
>     model="claude-opus-5-5", max_tokens=128000,
>     betas=["task-budgets-2026-03-13"],
>     output_config={"effort": "high",
>                    "task_budget": {"type": "tokens", "total": 64000}},
>     messages=[...], tools=[...],
> ) as stream:
>     response = stream.get_final_message()
> ```
>
> Il valore minimo di `total` è 20.000. È disponibile con Claude Fable 5.1 / Mythos 5.1, Fable 5 / Mythos 5, Opus 5.5, Opus 5 e Opus 4.8/4.7, ma **non** con Sonnet 5, Opus 4.6 o Haiku 4.5, né tramite Claude Code o Cowork (solo Messages API). Usalo in streaming: un valore elevato di `max_tokens` in una richiesta non in streaming può causare timeout HTTP. Dimensiona il budget in base alla distribuzione reale della durata delle attività: un budget palesemente insufficiente può indurre Claude a rifiutare, ridurre l'ambito o fermarsi in anticipo con un risultato parziale; può sembrare un rifiuto, ma non lo è. L'antipattern citato nell'esame resta valido: il budget regola il ritmo del loop, mentre `stop_reason` continua a terminarlo.

Chi esegue il loop: quattro modi per creare un agente

Due domande indipendenti distinguono le opzioni: **chi fornisce l'infrastruttura di esecuzione** (il loop e la gestione del contesto) e **chi fornisce l'ambiente di distribuzione** (l'infrastruttura su cui viene eseguito). È facile confondere Tool Runner e Claude Agent SDK perché entrambi forniscono solo l'infrastruttura di esecuzione: l'hosting resta a tuo carico.

Approccio	Cosa scrivi tu	Infrastruttura di esecuzione e distribuzione	Strumenti disponibili
Loop manuale (`client.messages.create`)	Il loop `while stop_reason == "tool_use"`	Crei tu l'infrastruttura di esecuzione e gestisci l'hosting	Solo gli strumenti che definisci
Tool Runner (`client.beta.messages.tool_runner`)	Solo le funzioni degli strumenti	L'SDK fornisce il loop; gestisci tu l'hosting	Solo gli strumenti che definisci

Approccio	Cosa scrivi tu	Infrastruttura di esecuzione e distribuzione	Strumenti disponibili
Managed Agents (beta, `/v1/agents` + `/v1/sessions`)	Configurazione dell'agente e risultati dei tuoi strumenti	Anthropic fornisce il loop **e** ospita un ambiente isolato per ogni sessione	Bash, file ed esecuzione di codice ospitati, oltre a Skills, MCP e ai tuoi strumenti
Claude Agent SDK (`claude-agent-sdk`)	Un prompt e le opzioni	Infrastruttura di Claude Code e strumenti integrati; gestisci tu l'hosting	Read/Write/Edit/Bash/Glob/Grep/WebSearch/WebFetch, oltre a MCP e subagenti

Tool Runner ≠ Claude Agent SDK. Tool Runner è incluso nell'SDK Anthropic standard (`anthropic` / `@anthropic-ai/sdk`) e gestisce il loop solo sugli strumenti che *definisci tu*: non include strumenti integrati, filesystem o ambiente isolato. Espone hook per ogni turno che consentono di gestire approvazioni, intercettare errori, modificare i risultati (ad esempio aggiungendo `cache_control`) e riprovare. Claude Agent SDK è Claude Code distribuito come libreria.

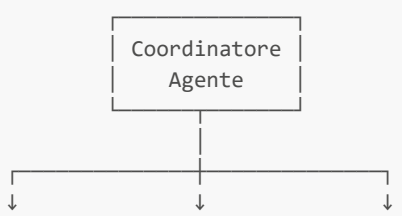
Managed Agents (beta) è la superficie più recente ed è l'unica che aggiunge una *distribuzione* gestita. Il flusso obbligatorio è: agente (creato una volta, versionato e salvato) → sessione (una per ogni esecuzione). I parametri `model`, `system` e `tools` appartengono all'**agente**, mai alla sessione. Ogni sessione prepara un container che funge da area di lavoro dell'agente e trasmette gli eventi; tu invii messaggi e risultati degli strumenti. Sono inoltre disponibili distribuzioni pianificate (sessioni avviate da cron), credenziali conservate in un archivio protetto e sostituite all'uscita, budget di sessione espressi in valuta ed elenchi di agenti. Intestazione beta: `managed-agents-2026-04-01`. Non è disponibile su Bedrock / Vertex / Foundry: lì usa Claude API e l'uso degli strumenti. Il flusso consigliato per il piano di controllo è ora la **CLI** `ant`: descrivi agenti, ambienti, skill, archivi di memoria e distribuzioni in file del repository, quindi esegui `ant apply` (v1.30.0, 2026-09-03). Il comando mostra un piano, lo applica e scrive un lockfile `claude-lock.json`: includilo nel commit affinché le esecuzioni successive, in locale o in CI, aggiornino le stesse risorse invece di crearne di nuove (`ant beta:agents create < agent.yaml` resta disponibile per operazioni singole). Il codice applicativo gestisce solo il piano dati (`sessions.create` con l'ID dell'agente salvato). `ant auth login` fornisce agli SDK anche un profilo OAuth, così un client `Anthropic()` standard funziona senza una chiave API nell'ambiente. Due novità di settembre: le policy di autorizzazione hanno acquisito `auto`; il server valuta ogni chiamata a strumenti dell'agente o MCP e la esegue, la nega se ad alto rischio oppure attende la tua approvazione, riportando il verdetto nel campo `evaluation` degli eventi `agent.tool_use`. Inoltre, `ant beta:sessions connect` collega il terminale a una sessione attiva per seguirla, inviare messaggi e approvare o negare le chiamate agli strumenti in attesa (`--web` rende disponibile localmente il visualizzatore delle sessioni della Console).

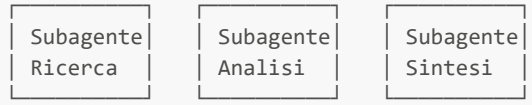
Scelta: usa il livello più semplice che funziona. Una singola chiamata o un flusso controllato dal codice è sufficiente per la maggior parte delle attività; scegli un agente solo quando l'attività è davvero aperta e guidata dal modello, il valore giustifica latenza e costo e gli errori sono rilevabili (test, revisione, rollback).

1.2 Orchestrazione multiagente

Architettura hub-and-spoke

Il modello standard per i sistemi multiagente è hub-and-spoke: un agente coordinatore gestisce più subagenti specializzati.





Responsabilità del coordinatore

Il coordinatore:

- **Scompone** l'attività in sottoattività
- **Delega** il lavoro ai subagenti appropriati
- **Instrada** tutte le comunicazioni tra subagenti (i subagenti non comunicano mai direttamente tra loro)
- **Aggrega** i risultati dei subagenti
- **Gestisce gli errori** dovuti ai malfunzionamenti dei subagenti
- **Valuta** la qualità della sintesi e delega nuovamente il lavoro se individua lacune

Isolamento dei subagenti

Concetto fondamentale: i subagenti hanno un contesto isolato. NON ereditano la cronologia della conversazione del coordinatore. Tutto ciò di cui un subagente ha bisogno deve essere incluso esplicitamente nel suo prompt.

Questo significa che:

- il coordinatore deve includere tutto il contesto pertinente nella descrizione dell'attività del subagente;
- i subagenti non possono fare riferimento alle parti precedenti della conversazione principale;
- ogni subagente inizia da zero e dispone solo delle informazioni fornite dal coordinatore.

Selezione dinamica dei subagenti

Non tutte le richieste richiedono tutti i subagenti. Il coordinatore dovrebbe:

- valutare ciò che la richiesta richiede davvero;
- invocare solo i subagenti pertinenti;
- evitare di instradare ogni richiesta attraverso una pipeline completa.

Esempio: una ricerca su un singolo argomento non richiede ricerca sul Web, analisi dei documenti e sintesi se basta una ricerca mirata.

Rischi della scomposizione

Scomposizione troppo ristretta: l'esame verifica esplicitamente questo rischio. Esempio: al coordinatore viene chiesto di fare ricerche sulle «industrie creative», ma le sottoattività riguardano solo le arti visive, trascurando musica, scrittura, cinema e design.

La soluzione consiste in una scomposizione iniziale più ampia e in controlli espliciti di copertura.

Cicli di perfezionamento iterativo

Il coordinatore valuta la sintesi e individua le lacune:

```
Coordinatore → Subagenti (primo ciclo)
        ↓
  Valuta i risultati
        ↓
  Individua le lacune
        ↓
Coordinatore → Subagenti (secondo ciclo, mirato)
        ↓
  Sintesi finale
```

1.3 Avvio dei subagenti e passaggio del contesto

Lo strumento Agent

I subagenti vengono avviati con lo strumento `Agent`, il nome ufficiale sia in Claude Code sia in Claude Agent SDK. Per avviare subagenti, il coordinatore deve includere `Agent` tra gli strumenti consentiti; è possibile limitare l'accesso a tipi specifici di subagente con `Agent(worker, researcher)`.

> **Nota sul nome:** il materiale meno recente (compresa la formulazione della guida d'esame) lo chiama strumento **Task**. `Task` era il nome originale dello stesso meccanismo in Claude Code; la documentazione attuale di Claude Code e dell'SDK usa `Agent`. Se una domanda d'esame cita lo «strumento Task», si riferisce a questo.
>
> **Nome dell'SDK:** il nome è cambiato da «Claude Code SDK» a **Claude Agent SDK**. Python: `pip install claude-agent-sdk`. TypeScript: `npm install @anthropic-ai/claude-agent-sdk`. Punto d'ingresso principale: `query()` + `ClaudeAgentOptions`.

Il contesto deve essere esplicito

Questo aspetto è oggetto di molte domande: **il contesto non viene ereditato automaticamente.** Quando avvia un subagente, il coordinatore deve includere nel prompt tutte le informazioni necessarie.

Da evitare:

```
"Analizza il problema del cliente"
# Il subagente non sa chi sia il cliente né quale sia il problema
```

Corretto:

```
"Il cliente n. 12345 segnala che l'ordine n. 67890 (effettuato il 2024-01-15,
per un totale di 149,99 $) è arrivato danneggiato. Richiede un rimborso completo.
Analizza la cronologia dell'ordine e verifica l'applicabilità della politica di reso."
```

Esecuzione parallela dei subagenti

Per eseguire subagenti in parallelo, invia più chiamate allo strumento `Agent` in **un'unica risposta del coordinatore**. Non distribuirle su turni separati: in quel caso l'esecuzione sarebbe sequenziale.

```
{
  "content": [
    {
      "type": "tool_use",
      "name": "Agent",
      "input": { "prompt": "Cerca articoli recenti su...", "subagent_type": "search" }
    },
    {
      "type": "tool_use",
      "name": "Agent",
      "input": { "prompt": "Analizza il documento in...", "subagent_type": "analysis" }
    }
  ]
}
```

Claude Code limita a 20 i subagenti simultanei (`CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`) e a 3 i livelli di avvio (`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`); raggiunto il limite di profondità, lo strumento `Agent` non è disponibile al subagente (in un fork resta nell'elenco, ma genera un errore invece di avviare un altro agente).

> **Oltre la guida d'esame: quando la ramificazione supera un singolo turno:** Claude Code dispone ora di quattro unità di orchestrazione; il modello hub-and-spoke dell'esame corrisponde alla prima. La differenza sta in *chi detiene il piano*.

	Subagenti	Skill	Team di agenti (sperimentale)	Workflow dinamici
Cosa sono	Un agente di lavoro avviato da Claude	Istruzioni seguite da Claude	Una sessione principale che supervisiona sessioni pari	Uno script JavaScript eseguito dal runtime
Chi decide cosa eseguire dopo	Claude, turno per turno	Claude, seguendo il prompt	L'agente principale	Lo script (`agent()`, `parallel()`, `pipeline()`)
Dove si trovano i risultati intermedi	Nel contesto di Claude	Nel contesto di Claude	In un elenco di attività condiviso	Nelle variabili dello script
Dimensioni	Alcune attività delegate per turno	Come i subagenti	Pochi agenti pari attivi a lungo	Da decine a centinaia di agenti per esecuzione, riprendibile

Un workflow trasferisce il piano del coordinatore dal modello al codice: è la soluzione deterministica al problema «il coordinatore ha dimenticato metà delle sottoattività» (§1.2). Claude scrive lo script; `/workflows` lo monitora; `ultracode` (un'impostazione di effort) consente a Claude di decidere quando usarne uno. I team di agenti richiedono `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`. La dimensione consigliata per un workflow medio è ora di 10 agenti (in precedenza 15), nei piani Pro il valore predefinito è «small» e `CLAUDE_CODE_WORKFLOW_MAX_CONCURRENT_AGENTS` (1–256) aumenta il limite di concorrenza per le esecuzioni con molti agenti che dipendono dall'inferenza.

AgentDefinition in Claude Agent SDK

```
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

async for message in query(
    prompt="Usa l'agente code-reviewer per esaminare questa codebase",
    options=ClaudeAgentOptions(
        allowed_tools=["Read", "Glob", "Grep", "Agent"],
        agents={
            "code-reviewer": AgentDefinition(
                description="Esperto nella revisione del codice Python e TypeScript",
                prompt="Analizza la qualità del codice, individua bug e suggerisci miglioramenti.",
                tools=["Read", "Glob", "Grep"],
            ),
            "test-writer": AgentDefinition(
                description="Specialista nella generazione di test",
                prompt="Scrivi test unitari completi per il codice fornito.",
                tools=["Read", "Write", "Bash"],
            ),
        },
    ),
):
    print(message)
```

I messaggi generati nel contesto di un subagente includono il campo `parent_tool_use_id`: usalo per attribuire i messaggi alla corretta esecuzione del subagente.

Fork e subagenti nuovi

Un **fork** offre il compromesso opposto rispetto a un subagente isolato: eredita *l'intera* conversazione (cronologia, prompt di sistema, strumenti e modello) e condivide la cache del prompt, quindi costa meno di un subagente nuovo. Solo il risultato finale torna nella conversazione principale; le chiamate agli strumenti restano isolate. I fork non possono avviare altri fork.

	Subagente nuovo	Fork
Cronologia della conversazione	Non ereditata: passa tutto esplicitamente	Ereditata per intero
Prompt di sistema / strumenti	Quelli del subagente	Gli stessi della sessione padre
Cache del prompt	Fredda	Condivisa con il padre (meno costosa)
Da usare per	Attività circoscritte e ben definite; esplorazioni che producono molto rumore	Rami divergenti a partire da una base condivisa

In Claude Code, Claude richiede un fork tramite lo strumento `Agent` con `subagent_type: "fork"` (la modalità fork è attiva per impostazione predefinita nelle sessioni interattive e i subagenti avviati in questo modo vengono eseguiti in background). Puoi avviarne uno manualmente con `/subtask <prompt>`. Due comandi simili sono facili da confondere: `/fork` ora *copia* l'intera sessione in una sessione separata in background (vista degli agenti), mentre `/branch` passa a una copia della conversazione per provare un approccio diverso. SDK e CLI espongono anche `--fork-session`, che riprende una sessione con un nuovo ID; `fork_session` è il vecchio nome a livello API per la stessa idea di «creare un ramo da una base condivisa».

Passaggio strutturato del contesto

Buona pratica: separa contenuti e metadati quando passi il contesto tra agenti.

```
{
  "findings": [
    {
      "claim": "I ricavi sono aumentati del 15% su base annua",
      "evidence": "Relazione finanziaria del quarto trimestre, pagina 3",
      "source_url": "https://example.com/earnings",
      "document_name": "Relazione finanziaria del quarto trimestre 2024",
      "publication_date": "2025-01-15"
    }
  ]
}
```

In questo modo si preserva l'attribuzione e si consente all'agente ricevente di valutare la qualità della fonte.

1.4 Applicazione delle regole del workflow e passaggio di consegne

La distinzione fondamentale

È uno dei concetti più frequentemente verificati nell'esame:

Tipo di applicazione	Quando usarlo	Livello di garanzia
Programmatica (hook, prerequisiti)	Conseguenze finanziarie o di sicurezza	Deterministica: applicazione al 100%
Basata sul prompt (istruzioni)	Quando è accettabile fare del proprio meglio	Probabilistica: il rischio di errore non è nullo

Esempio: operazioni finanziarie

Scenario: un agente del supporto clienti deve verificare l'identità prima di elaborare un rimborso.

Approccio errato (basato sul prompt):

```
"Verifica sempre l'identità del cliente prima di elaborare qualsiasi rimborso."
```

Funzionerà nella maggior parte dei casi, ma il rischio di errore non è nullo. Nelle operazioni finanziarie, «nella maggior parte dei casi» non è sufficiente.

Approccio corretto (programmatico): Un hook `PreToolUse` su `process_refund` verifica che `get_customer` sia stato chiamato e abbia restituito un ID cliente verificato. In caso contrario, l'hook blocca il rimborso con codice di uscita 2.

Passaggio di consegne a una persona

Quando un agente inoltra un caso a una persona, quest'ultima in genere NON ha accesso alla trascrizione completa della conversazione. Il passaggio di consegne deve includere un riepilogo strutturato:

```
Cliente: Jane Doe (n. 12345)
Problema: prodotto ricevuto danneggiato
Ordine: n. 67890, 149,99 $, effettuato il 2024-01-15
Causa principale: danno durante la spedizione
Azioni intraprese: ordine verificato, danno confermato tramite fotografie
Azione consigliata: rimborso completo e invio di un prodotto sostitutivo
Importo del rimborso: 149,99 $
```

1.5 Hook di Claude Code / Agent SDK

Eventi degli hook (catalogo attuale)

Nel 2026 il sistema degli hook si è ampliato notevolmente: il catalogo attuale comprende **33 eventi** suddivisi in otto gruppi. Gli eventi rilevanti per l'esame sono contrassegnati con ★.

Gruppo	Eventi
Configurazione iniziale	`Setup` (viene eseguito una volta per sessione)
Sessione e turno	`SessionStart` ★, `SessionEnd` ★, `UserPromptSubmit` ★, `UserPromptExpansion`, `Stop` ★, `StopFailure`
Strumenti e loop agentico	`PreToolUse` ★, `PostToolUse` ★, `PostToolUseFailure`, `PostToolBatch`, `PermissionRequest` ★, `PermissionDenied`
Agente e attività	`SubagentStart`, `SubagentStop` ★, `TaskCreated`, `TaskCompleted`, `TeammateIdle`
File e configurazione	`FileChanged`, `CwdChanged`, `DirectoryAdded`, `ConfigChange`, `InstructionsLoaded`
Compattazione	`PreCompact` ★, `PostCompact`
Modello	`PreModelSwitch`, `PostModelSwitch` (blocca, conferma o annota un cambio di modello; novità della versione v2.1.251)
Contesto e worktree	`Notification`, `MessageDisplay`, `Elicitation`, `ElicitationResult`, `WorktreeCreate`, `WorktreeRemove`

Schema generale del ciclo di vita da ricordare per l'esame:

```
SessionStart
  → UserPromptSubmit
    → [Loop agentico]:
        PreToolUse
          → PermissionRequest  (può attivare PermissionDenied)
            → PostToolUse  /  PostToolUseFailure
              → SubagentStart / SubagentStop
                → TaskCreated / TaskCompleted
    → Stop  /  StopFailure
  → PreCompact → PostCompact
  → PreModelSwitch → PostModelSwitch   (solo quando cambia il modello)
→ SessionEnd
```

Tipi di hook

Sono supportati cinque tipi. `mcp_tool` è una novità del 2026.

Tipo	Funzionamento	Caso d'uso
`command`	Esegue un comando shell. Codice di uscita 0 = consenti, codice 2 = blocca	Convalida dei file, controlli di linting
`http`	Invia una richiesta POST a un endpoint URL	Registrazione esterna degli audit, webhook
`mcp_tool`	Chiama uno strumento su un server MCP configurato	Convalida strutturata tramite un servizio riutilizzabile
`prompt`	Una singola chiamata LLM restituisce una decisione sì/no	Approvazione dipendente dal contesto
`agent`	Subagente con accesso agli strumenti e più turni	Verifiche di conformità complesse

Due vincoli introdotti nelle versioni recenti: `PermissionRequest` non esegue più hook di tipo `agent` (la loro risposta non poteva consentire né negare la richiesta), quindi da v2.1.280+ usa `command` o `http` per questo evento; inoltre, gli hook `mcp_tool` associati a eventi bloccanti attendono che il server MCP completi la connessione, invece di essere ignorati.

Hook PreToolUse (struttura JSON attuale)

Ogni matcher contiene un **array** di `hooks` (il vecchio campo singolare `hook` non esiste più). Gli hook possono essere `async`, subordinati alle regole di autorizzazione `if` e includere un `statusMessage` visualizzato nell'interfaccia.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "process_refund",
        "hooks": [
          {
            "type": "command",
            "if": "Bash(git *)",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/check-refund-authorization.sh",
            "timeout": 600,
            "statusMessage": "Verifica dell'autorizzazione al rimborso...",
            "shell": "bash"
          },
          {
            "type": "mcp_tool",
            "server": "compliance",
            "tool": "check_refund_policy",
            "input": { "amount": "${tool_input.amount}" },
            "timeout": 60
          }
        ]
      }
    ]
  },
  "disableAllHooks": false
}
```

Output di controllo della decisione (invariato):

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "L'identità del cliente non è ancora stata verificata",
    "additionalContext": "Chiama prima get_customer per verificare l'identità"
```

```
    }
  }
```

Valori di `permissionDecision`: `"allow"` (consenti), `"deny"` (nega), `"ask"` (chiedi).

Hook PostToolUse

Intercettano i risultati degli strumenti prima che Claude li elabori. Sono utili per:

- normalizzare timestamp, formati di data e codici di stato;
- rimuovere dai risultati i dati sensibili;
- aggiungere metadati ai risultati.

Se lo strumento stesso genera un errore, viene attivato `PostToolUseFailure`: gestisci lì i nuovi tentativi e la diagnostica.

Hook in skill e subagenti

Ora è possibile associare gli hook al **ciclo di vita di una skill** tramite il campo frontmatter `hooks:` in `SKILL.md`; anche i subagenti possono avere definizioni di hook proprie. In questo modo puoi applicare regole locali (ad esempio, una skill `commit` esegue `cargo fmt` prima di `git commit`) senza appesantire le impostazioni dell'intero progetto.

Percorsi di configurazione degli hook

Percorso	Ambito	Condiviso?
`~/.claude/settings.json`	Tutti i progetti dell'utente	No (personale)
`.claude/settings.json`	Questo progetto	Sì (controllo versione)
`.claude/settings.local.json`	Questo progetto, su questo computer	No (escluso da Git)
Impostazioni delle policy gestite	Intera organizzazione	Sì (gestito dall'amministratore)

1.6 Strategie di scomposizione delle attività

Concatenazione di prompt (sequenza fissa)

Per attività prevedibili e articolate in più aspetti, quando i passaggi sono noti in anticipo:

```
Passaggio 1: analizza ogni file singolarmente (analisi locale)
Passaggio 2: verifica l'integrazione tra i file (flusso dei dati, coerenza)
Passaggio 3: genera il rapporto finale
```

L'output di ogni passaggio alimenta quello successivo. La sequenza è prestabilita.

Scomposizione dinamica e adattiva

Per indagini aperte, in cui le sottoattività emergono dalle scoperte:

```
Richiesta iniziale: "Perché l'app è lenta?"
  → Analizza l'utilizzo della CPU
    → Scoperta: le query al database sono il collo di bottiglia
      → Analizza gli schemi delle query
        → Scoperta: query N+1 nell'endpoint che elenca gli utenti
          → Genera una correzione per le query specifiche
```

Le sottoattività vengono definite in base a ciò che emerge da ogni passaggio.

Architettura di revisione in più passaggi

Per la revisione del codice di PR di grandi dimensioni:

Passaggio 1: analisi locale di ogni file

- Ogni file viene esaminato in modo indipendente.
- L'analisi mantiene lo stesso livello di approfondimento per tutti i file.
- Individua i problemi locali (bug, stile, tipi).

Passaggio 2: integrazione tra i file

- Un passaggio separato esamina le interazioni tra i file.
- Individua problemi nel flusso dei dati, incompatibilità tra interfacce e incoerenze tra file.
- Usa un prompt distinto, incentrato sugli aspetti di integrazione.

Perché due passaggi? La revisione in un unico passaggio di PR di grandi dimensioni può causare:

- dispersione dell'attenzione (il modello si concentra sui primi file e dedica meno attenzione ai successivi);
- risultati contraddittori (criteri incoerenti tra i file);
- problemi di integrazione non rilevati (esaminando un file alla volta non si vedono i problemi che coinvolgono più file).

1.7 Gestione delle sessioni

Ripresa delle sessioni

`--resume <session-name>` riprende una conversazione precedente identificata da un nome, mantenendone l'intero contesto.

Quando riprendere:

- durante un'indagine in corso, se i risultati degli strumenti sono ancora validi;
- per lo sviluppo iterativo all'interno della stessa sessione.

Quando iniziare una nuova sessione:

- quando i risultati degli strumenti non sono più aggiornati (i file sono cambiati, i servizi sono stati riavviati);
- quando il contesto si è allontanato troppo dall'attività corrente;
- in questi casi, passa un riepilogo strutturato dei risultati precedenti anziché riprendere la sessione.

Creazione di un fork della sessione

`--fork-session` riprende una sessione esistente usando un **nuovo** ID: l'originale resta invariata e ogni ramo può esplorare un approccio diverso senza influenzare gli altri. In una sessione interattiva, `/subtask <prompt>` avvia un fork che eredita l'intera conversazione e ne restituisce solo il risultato; `/fork` copia la sessione in una sessione separata in background; `/branch` passa a una copia (vedi §1.3).

Caso d'uso: provare due diverse strategie di refactoring partendo dalla stessa situazione.

Spostamento di una sessione tra ambienti

Le sessioni non sono più legate a un singolo computer. `claude --cloud "<task>"` avvia il lavoro in una sessione cloud (`--remote` è un alias deprecato); `claude --teleport` riporta una sessione Web nel terminale; `/desktop` trasferisce la sessione corrente del terminale all'app desktop per esaminare visivamente le differenze. Remote Control permette di controllare da un telefono o da un altro browser una sessione locale in esecuzione. In locale, `claude --bg "<task>"` avvia una **sessione in background** e restituisce subito il controllo; `claude agents` apre la vista degli agenti, mentre `claude attach / logs / stop / respawn / rm <id>` consente di gestire la sessione dalla shell (`respawn --all` riavvia tutte le sessioni attive, ad esempio per caricare un nuovo binario). Questo è importante dal punto di vista architetturale: una «sessione» è un'unità di stato portatile, quindi tutto ciò che l'agente non deve perdere va salvato in file o in memoria, non nella cronologia scorrevole del terminale.

Aggiornamento delle sessioni riprese

Quando riprendi una sessione dopo aver modificato dei file, devi dire esplicitamente a Claude che cosa è cambiato:

```
"Dall'ultima sessione ho aggiornato auth.py per usare token JWT al posto
dei cookie di sessione. Analizza di nuovo il flusso di autenticazione tenendo conto di queste
modifiche."
```

Non dare per scontato che la sessione ripresa rilevi da sola le modifiche: i risultati degli strumenti della sessione precedente potrebbero basarsi su contenuti dei file non più aggiornati.